INTERNATIONAL ISLAMIC UNIVERSITY MALAYSIA

KULLIYYAH OF ENGINEERING

MECHATRONICS SYSTEM INTEGRATION (MCTA 3203)

SEMESTER 2 2025/2026

WEEK 6: SMART SURVELLANCE CAMERA USING ESP32-CAM

SECTION 1

GROUP 11

INSTRUCTOR: DR. WAHJU SEDIONO & ASSOC. PROF. EUR. ING. IR. TS. GS. INV. DR. ZULKIFLI BIN ZAINAL ABIDIN

Date of Experiment: Wednesday, 8th April 2026

Date of Submission: Wednesday, 15th April 2026

NO.	NAME	MATRIC NO.
1.	NURUL ANIS BINTI MOHD KHAIRI	2419788
2.	WAN NUR ATHIRAH BINTI MEOR MOHD REDZUAN	2320102
3.	SITI NURKHALISAH BINTI NAHZAMRI	2328482

ABSTRACT

This experiment demonstrates the integration of an ESP32-CAM with a servo motor and a push button to create a manually controlled panning surveillance system. Utilizing the ESP32's Wi-Fi capabilities, the system initializes a web server to provide a real-time HTTP video stream accessible via a local IP address. The hardware logic employs an internal pull-up resistor configuration, where the servo motor executes a continuous 180-degree horizontal sweep only when the push button is depressed (GPIO 13 reading LOW). When released, the motor remains stationary as the voltage returns to a HIGH state. The results confirm the successful implementation of conditional loop logic and PWM signal generation, highlighting the project's viability for IoT-based home automation and wireless security applications.

TABLE OF CONTENTS

INTRODUCTION 5
Task 1: 6
1.0 MATERIALS AND EQUIPMENTS 6
1.1 EXPERIMENTAL SETUP 6
1.2 METHODOLOGY 7
- **a. Circuit assembly** 7
- **b. Circuit logic** 7
- **c. Programming and uploading.** 7
- **d. Programming Logic** 7
- **e. Programming code.** 8

1.3 DATA COLLECTION 11
1.4 DATA ANALYSIS 11
1.5 RESULT 12
1.6 DISCUSIONS 12
CONCLUSION 13
RECOMMENDATIONS 13
REFERENCES 13
ACKNOWLEDGEMENT 14
STUDENTS DECLARATION 14
- **Certificate of Originality and Authenticity** 14

INTRODUCTION

The rapid advancement of the Internet of Things (IoT) has revolutionized modern security, enabling the development of low-cost, high-performance wireless surveillance systems. The ESP32-CAM module stands at the forefront of this movement, combining a powerful microcontroller with an integrated camera and Wi-Fi connectivity. By leveraging web server protocols, these devices can stream live video data over a network, allowing users to monitor environments remotely through standard web browsers. Such systems are fundamental to the growth of smart home ecosystems, providing scalable solutions for home automation, smart doorbells, and portable security monitoring.

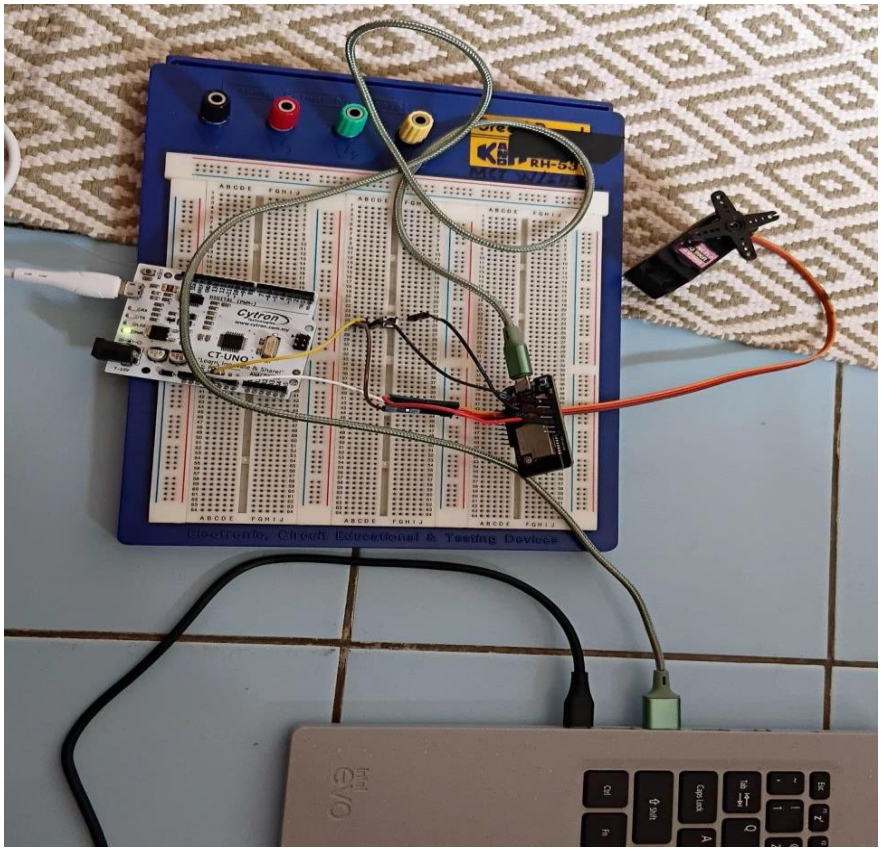
In this experiment, a servo-assisted camera mount was developed to address the need for manual panning control. The system was designed to remain idle until a physical push button is activated, triggering a conditional loop that manages the servo motor's position. By connecting the ESP32-CAM to a local network and interfacing it with a push button on GPIO 13, the experiment explores the relationship between digital input states (HIGH/LOW) and Pulse Width Modulation (PWM) output. The primary objective was to successfully synchronize the live video feed with a mechanical 180-degree sweep, demonstrating a practical application of interrupt-style manual control in IoT hardware.

Task 1: <u>Integrate a pushbutton for manual control of servo panning.</u>

1.0 <u>MATERIALS AND EQUIPMENTS</u>

- ESP32-CAM Module version OV2640
- CH340 USB to TTL Serial Cable or Micro-USB Cable or FTDI Cable
- Servo Motor
- Pushbutton
- 5V Power Supply (2A) or Power bank (5V 2A)
- Jumper wires
- Breadboard

1.1 <u>EXPERIMENTAL SETUP</u>



<u>Fig 1 shows the connection of ESP32-CAM Module version OV2640 with servo and cytron CT-UNO.</u>

1) Bridge the IO pin to the gnd.
2) The PWM of servo motor is connected to the ESP 32-CAM.
3) The positive supply for servo motor is connected to 5v of Cytron CT uno.
4) The negative supply of servo motor is connected to the GND of Cytron CT-UNO.
5) The one leg of the pushbutton is connected to a GPIO 13, and the other leg of the push button is connected with the GND.

1.2 METHODOLOGY

a. Circuit assembly

- Connect the components as shown in the Fig 1.

b. Circuit logic

- When the button is not pressed the servo motor will not move.
- When the button is pressed the servo motor will rotate.
- The servo motor will rotate from 0 to 180 and it will rotate continuously left-right panning motion.

c. Programming and uploading.

- Download the ESP32 version 2.0.14 (by Espressif systems) from the board manager located on the left side of the IDE
- Download the ESP32 version 2.0.10 (by Arduino) from the board manager located on the left side of the IDE

d. Programming logic

- const char *ssid = "Anis's" and const char *password = "00000000" will stores the WIFI network name and password and the ESP32-CAM will connect to the internet for live streaming access.
- The purpose of Servo myServo; int posisi = 0; int arah = 1; is to stores the current angle which is from 0 to 180 degrees and the +1 is to increase the angle and the -1 to decrease the angle.
- if (digitalRead(PIN_BUTANG) == LOW) will checked whether the button is pressed and LOW means the button is actively pressed.
- if (posisi >= 180 || posisi <= 0) { arah *= -1;} shows that when the servo motor reached 180 or 0 the direction will reverse and it will create a continuous back-and-forth planning motion.

e. **Programming code.**

```
#include "esp_camera.h"
#include <WiFi.h>
#include <ESP32Servo.h> // Library untuk Servo

// ============================
// Select camera model
// ============================
#define CAMERA_MODEL_AI_THINKER // PENTING: Untuk ESP32-CAM
AI-Thinker
#include "camera_pins.h"

// ============================
// WiFi Credentials
// ============================
const char *ssid = "Anis's";
const char *password = "00000000";

// ============================
// Servo & Button Configuration
// ============================
const int PIN_SERVO = 14;
const int PIN_BUTANG = 13; // Sesuai arahan tugasan

Servo myServo;
int posisi = 0;
int arah = 1;

void startCameraServer();
void setupLedFlash();

void setup() {
  Serial.begin(115200);
  Serial.setDebugOutput(true);
  Serial.println();

  // --- Setup Pushbutton & Servo ---
  pinMode(PIN_BUTANG, INPUT_PULLUP); // Gunakan internal pull-up
  ESP32PWM::allocateTimer(3);        // Allocate timer untuk PWM
  myServo.setPeriodHertz(50);        // Servo standard 50Hz
  myServo.attach(PIN_SERVO, 500, 2400);
  myServo.write(posisi);
  // -------------------------------
```

```
camera_config_t config;
config.ledc_channel = LEDC_CHANNEL_0;
config.ledc_timer = LEDC_TIMER_0;
config.pin_d0 = Y2_GPIO_NUM;
config.pin_d1 = Y3_GPIO_NUM;
config.pin_d2 = Y4_GPIO_NUM;
config.pin_d3 = Y5_GPIO_NUM;
config.pin_d4 = Y6_GPIO_NUM;
config.pin_d5 = Y7_GPIO_NUM;
config.pin_d6 = Y8_GPIO_NUM;
config.pin_d7 = Y9_GPIO_NUM;
config.pin_xclk = XCLK_GPIO_NUM;
config.pin_pclk = PCLK_GPIO_NUM;
config.pin_vsync = VSYNC_GPIO_NUM;
config.pin_href = HREF_GPIO_NUM;
config.pin_sccb_sda = SIOD_GPIO_NUM;
config.pin_sccb_scl = SIOC_GPIO_NUM;0
config.pin_pwdn = PWDN_GPIO_NUM;
config.pin_reset = RESET_GPIO_NUM;
config.xclk_freq_hz = 20000000;
config.frame_size = FRAMESIZE_UXGA;
config.pixel_format = PIXFORMAT_JPEG;
config.grab_mode = CAMERA_GRAB_WHEN_EMPTY;
config.fb_location = CAMERA_FB_IN_PSRAM;
config.jpeg_quality = 12;
config.fb_count = 1;

if (config.pixel_format == PIXFORMAT_JPEG) {
  if (psramFound()) {
    config.jpeg_quality = 10;
    config.fb_count = 2;
    config.grab_mode = CAMERA_GRAB_LATEST;
  } else {
    config.frame_size = FRAMESIZE_SVGA;
    config.fb_location = CAMERA_FB_IN_DRAM;
  }
}

// Camera init
esp_err_t err = esp_camera_init(&config);
if (err != ESP_OK) {
  Serial.printf("Camera init failed with error 0x%x", err);
```

```
    return;
  }

  sensor_t *s = esp_camera_sensor_get();
  if (s->id.PID == OV3660_PID) {
    s->set_vflip(s, 1);
    s->set_brightness(s, 1);
    s->set_saturation(s, -2);
  }
  if (config.pixel_format == PIXFORMAT_JPEG) {
    s->set_framesize(s, FRAMESIZE_QVGA);
  }

#if defined(LED_GPIO_NUM)
  setupLedFlash();
#endif

  WiFi.begin(ssid, password);
  WiFi.setSleep(false);

  Serial.print("WiFi connecting");
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("\nWiFi connected");

  startCameraServer();

  Serial.print("Camera Ready! Use 'http://");
  Serial.print(WiFi.localIP());
  Serial.println("' to connect");
}

void loop() {
  // --- Mandatory Task: Manual Control Panning ---
  if (digitalRead(PIN_BUTANG) == LOW) {
    Serial.println("Butang dikesan! Menggerakkan servo...");
    // ... kod gerak servo ...
  }

  // Servo hanya bergerak bila butang ditekan (LOW)
  if (digitalRead(PIN_BUTANG) == LOW) {
```

```
    posisi += arah;

    // Pusing balik bila sampai had 180 atau 0
    if (posisi >= 180 || posisi <= 0) {
      arah *= -1;
    }

    myServo.write(posisi);
    delay(15); // Kelajuan panning. Kecilkan nombor untuk lebih laju.
  }
}
```

1.3 DATA COLLECTION

PUSH BUTTON	SERVO MOTOR
When the push button is being pressed.	Servo motor will rotate continuously left-right panning motion.
When the push button is not being pressed.	Servo motor will stop rotating.

1.4 DATA ANALYSIS

From the data collection we can see that when the push button is being pushed, the servo motor will rotate continuously left-right panning motion, this is because the servo motor is moving in a conditional loop. When the GPIO 13 is LOW (0), the servo will move 180-degree sweep because overpowers the pull-up resistor. However, when the push button isn't being pushed, the internal resistance will pull the voltage up to 3.3V this results in a digital reading of HIGH (1).

1.5 RESULT

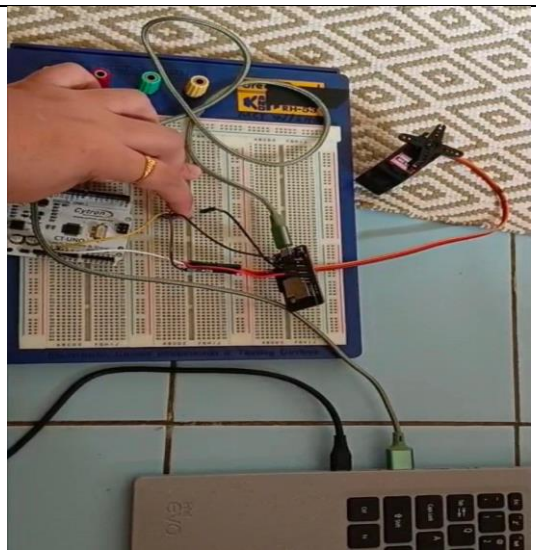


Fig 2 shows the servo is rotating when the button is being pushed.

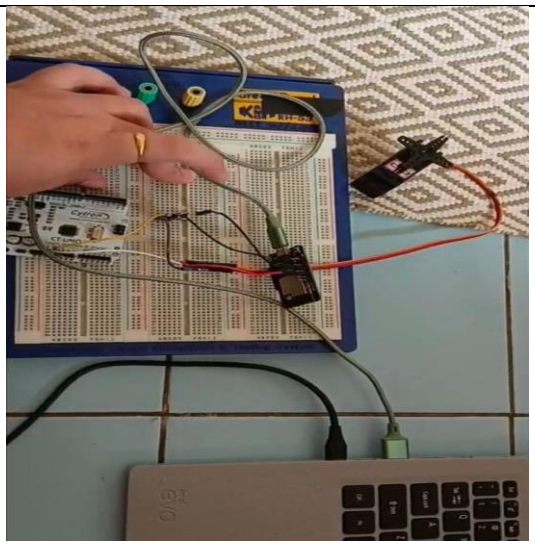


Fig 3 shows the servo is not rotating when the button is not being pushed.

1.6 DISCUSIONS

This task successfully demonstrates us how the ESP32-CAM being combined effectively withs servo motor and a push button. The main objective for this task is to satisfy the need for manual control panning by enabling camera to rotate horizontally only when the push button is pressed. The code of const char *ssid = "Anis's" and const char *password = "00000000" is being used because the working principle of ESP 32-CAM needs to connect to the Wi-fi network using predefined credentials and it will initialize the camera module and starts the web server. The user will access the IP address of ESP32-CAM on a browser to view the live camera feed, and it will be streamed in real lifetime using http. The application of ESP 32-CAM is it can be wireless surveillance and security camera, IOT based home automation and smart doorbell system.

The logic of "if (digitalRead(PIN_BUTANG) == LOW){posisi += arah; myServo.write(posisi);}" states that when the push button is pressed, the ESP 32-CAM uses digitalRead() to read the input button. It then uses the function of myServ.write(posisi) to send a PWM signal to a servo motor, which causes the servo to rotate to the desired angle. This process is repeated when the button state changes.

CONCLUSION

The experiment successfully achieved its objective of creating a responsive, manual panning camera system using the ESP32-CAM. The data confirms that the use of an internal pull-up resistor effectively managed the input logic, where a LOW signal successfully overrode the default 3.3V HIGH state to trigger the servo's rotation. Through the application of myServo.Write() functions within a conditional loop, the system provided smooth horizontal movement while simultaneously maintaining a stable HTTP live stream. This project highlights the effectiveness of combining simple digital inputs with complex Wi-Fi modules, proving that the ESP32-CAM is a robust platform for versatile, real-time IoT applications like smart surveillance and automated home security.

RECOMMENDATIONS

Adding a second servo motor to produce a vertical tilt movement is one suggestion we may make for this work. This would enhance the monitoring coverage and result in a complete pan-tilt surveillance system. For example, a pan tilt system using two SG90 servos will improve surveillance because it helps the camera to move in multiple directions, it can move p, down, left and right. Therefore, the camera can be used to observe floor level movement and higher area such as windows and doors.

The second suggestion is to incorporate facial detection or object tracking instead manually control using the push button so that the camera can identify movement without requiring input from the user. Using image processing and face detections algorithms, the face will be identified and detected even when the user is far away from the camera. For an instance, if the face moves to the left, the servo will rotate to the left and if the face moves to the right, the camera will follow automatically. This will eventually make the system smarter and suitable for modern security.

REFERENCES

Edge AI : Face Tracking Camera using ESP32 | Embedded systems. (2026). Iisc.ac.in. https://labs.dese.iisc.ac.in/embeddedlab/edge-ai-face-tracking-camera-using-esp32/

ESP32-CAM Pan and Tilt Video Streaming Web Server | Random Nerd Tutorials. (2021, April 8). Random Nerd Tutorials. https://randomnerdtutorials.com/esp32-cam-pan-and-tilt-2-axis/

ESP32IO.com. (2025). ESP32 - Controls Servo Motor via Web | ESP32 Tutorial. ESP32 Tutorial. https://esp32io.com/tutorials/esp32-controls-servo-motor-via-web

ESP32IO.com. (2026). ESP32 - Button - Servo Motor | ESP32 Tutorial. ESP32 Tutorial. https://esp32io.com/tutorials/esp32-button-servo-motor

Team, S. (2026). ESP32-CAM Based Wireless Surveillance Camera Using Wi-Fi | ESP32 Project Guide. Sarkitshala.com. https://www.sarkitshala.com/esp/cam-wireless-camera

ACKNOWLEDGEMENT

We would like to sincerely thank the instructors, Assoc. Prof. Eur. Ing. Ir. Ts. Gs. Inv. Dr. Zulkifli Bin Zainal Abidin and the lab technicians, for all of their help, encouragement and support during this project. Their knowledge and perceptions have greatly influenced the course of this work. Additionally, we would like to express our gratitude to our peers for their support and cooperation, both of which were crucial to the accomplishment of this project.

STUDENTS DECLARATION

We hereby declare that, except for the places where it is acknowledged, all of the work presented in this report is entirely ours. We certify that, in completing this project, we have complied with the standards of academic integrity and have not engaged in any plagiarism or unethical behaviour. Every information and support source used in this work has been appropriately referenced and acknowledged.

Certificate of Originality and Authenticity

This is to certify that we are responsible for the work submitted in this report, that the original work is our own except as specified in the references and acknowledgement, and that the original work contained herein has not been done by unspecified sources or persons. We hereby certify that this report has not been done by only one individual, and all of us have contributed to the report. The length of contribution to the reports by each individual is noted within this certificate. We also hereby certify that we have read and understand the content of the total report and no further improvement on the reports is needed from any of the individual's contributor to the report. We therefore agreed unanimously that this report shall be submitted for marking, and this final report has been verified by us.

Signature: *nurulanis*	Read [/]
Name: NURUL ANIS BINTI MOHD KHAIRI	Understand[/]
Matric Number: 2419788	Agree [/]
Signature: *athirahmeor*	Read [/]
Name: WAN NUR ATHIRAH BINTI MEOR MOHD REDZUAN	Understand[/]
Matric Number: 2320102	Agree [/]
Signature: *nurkhalisah*	Read [/]
Name: SITI NURKHALISAH BINTI NAHZAMRI	Understand[/]
Matric Number: 2328482	Agree [/]